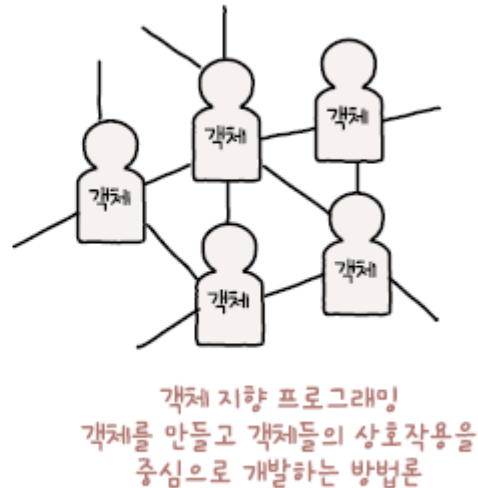


클래스

1. 클래스의 기본

시작하기 전에

- 객체 지향 프로그래밍(Object Oriented Programming)
 - 객체(object)를 우선으로 생각해서 프로그래밍하는 것
 - 클래스(class) 기반의 객체 지향 프로그래밍 언어는 클래스를 기반으로 객체 만들고, 그러한 객체를 우선으로 생각하여 프로그래밍함



객체

- 데이터(data) -> 객체
 - 예시 - 객체 만들기

```
01  # 학생 리스트를 선언합니다.
02  students = [
03      { "name": "윤인성", "korean": 87, "math": 98, "english": 88, "science": 95 },
04      { "name": "연하진", "korean": 92, "math": 98, "english": 96, "science": 98 },
05      { "name": "구지연", "korean": 76, "math": 96, "english": 94, "science": 90 },
06      { "name": "나선주", "korean": 98, "math": 92, "english": 96, "science": 92 },
07      { "name": "윤아린", "korean": 95, "math": 98, "english": 98, "science": 98 },
08      { "name": "윤명월", "korean": 64, "math": 88, "english": 92, "science": 92 }
09  ]
10
11  # 학생을 한 명씩 반복합니다.
12  print("이름", "총점", "평균", sep="\t")
```

객체

```
13 for student in students:
14     # 점수의 총합과 평균을 구합니다.
15     score_sum = student["korean"] + student["math"] + \
16         student["english"] + student["science"]
17     score_average = score_sum / 4
18     # 출력합니다.
19     print(student["name"], score_sum, score_average, sep="\t")
```

실행결과

이름	총점	평균
윤인성	368	92.0
연하진	384	96.0
구지연	356	89.0
나선주	378	94.5
윤아린	389	97.25
윤명월	336	84.0

객체

- 객체(object)

- 여러 가지 속성 가질 수 있는 모든 대상
- 예시 – 객체를 만드는 함수(1)

```
01  # 딕셔너리를 리턴하는 함수를 선언합니다.
02  def create_student(name, korean, math, english, science):
03      return {
04          "name": name,
05          "korean": korean,
06          "math": math,
07          "english": english,
08          "science": science
09      }
10
```

객체

```
11  # 학생 리스트를 선언합니다.
12  students = [
13      create_student("윤인성", 87, 98, 88, 95),
14      create_student("연하진", 92, 98, 96, 98),
15      create_student("구지연", 76, 96, 94, 90),
16      create_student("나선주", 98, 92, 96, 92),
17      create_student("윤아린", 95, 98, 98, 98),
18      create_student("윤명월", 64, 88, 92, 92)
19  ]
20
21  # 학생을 한 명씩 반복합니다.
22  print("이름", "총점", "평균", sep="\t")
23  for student in students:
24      # 점수의 총합과 평균을 구합니다.
25      score_sum = student["korean"] + student["math"] + \
26          student["english"] + student["science"]
27      score_average = score_sum / 4
28      # 출력합니다.
29      print(student["name"], score_sum, score_average, sep="\t")
```

객체

- 학생을 매개변수로 받는 형태의 함수로 만들면 코드가 더 균형 잡힘

```
01  # 딕셔너리를 리턴하는 함수를 선언합니다.
02  def create_student(name, korean, math, english, science):
03      return {
04          "name": name,
05          "korean": korean,
06          "math": math,
07          "english": english,
08          "science": science
09      }
10
11  # 학생을 처리하는 함수를 선언합니다.
12  def student_get_sum(student):
13      return student["korean"] + student["math"] + \
14          student["english"] + student["science"]
15
16  def student_get_average(student):
17      return student_get_sum(student) / 4
18
19  def student_to_string(student):
20      return "{}\t{}\t{}".format(
21          student["name"],
22          student_get_sum(student),
23          student_get_average(student))
```

→ 01~23행까지
학생 객체와
관련된 부분

객체

```
24
25 # 학생 리스트를 선언합니다.
26 students = [
27     create_student("윤인성", 87, 98, 88, 95),
28     create_student("연하진", 92, 98, 96, 98),
29     create_student("구지연", 76, 96, 94, 90),
30     create_student("나선주", 98, 92, 96, 92),
31     create_student("윤아린", 95, 98, 98, 98),
32     create_student("윤명월", 64, 88, 92, 92)
33 ]
34
35 # 학생을 한 명씩 반복합니다.
36 print("이름", "총점", "평균", sep="\t")
37 for student in students:
38     # 출력합니다.
39     print(student_to_string(student))
```

→ 25~39행까지
객체를 활용하는
처리

클래스 선언하기

- 클래스(class)

- 객체를 조금 더 효율적으로 생성하기 위해 만들어진 구문

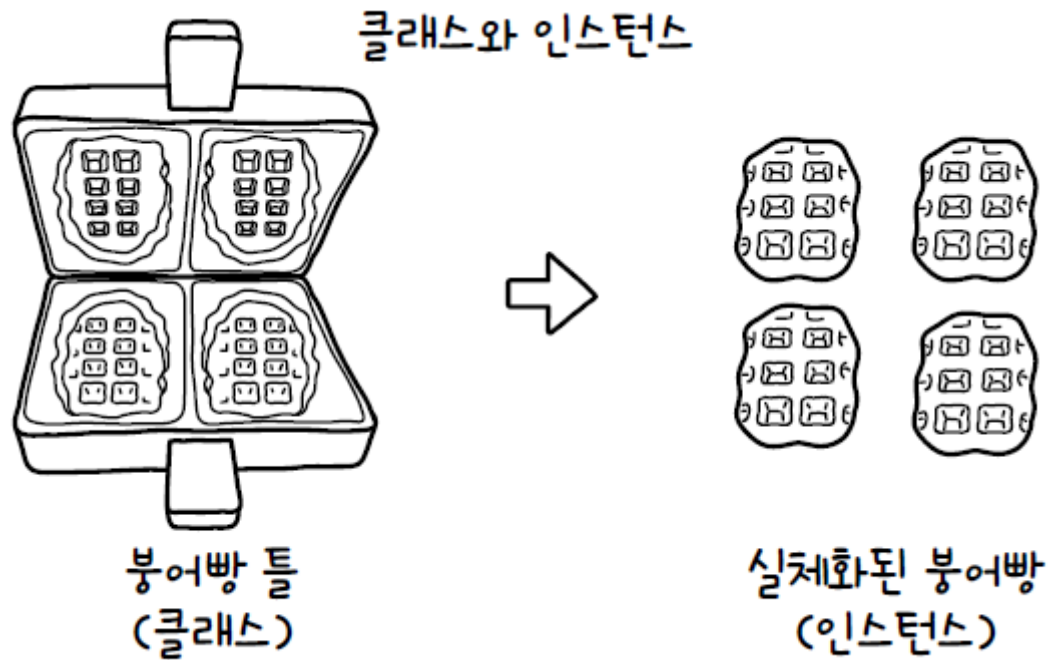
```
class 클래스 이름:  
    클래스 내용
```

- 인스턴스(instance)

- 생성자 사용하여 클래스 기반으로 만들어진 객체

```
인스턴스 이름(변수 이름) = 클래스 이름() → 생성자 함수라고 부릅니다.
```

클래스 선언하기



클래스 선언하기

- 클래스를 기반으로 학생 6명을 선언

```
# 클래스를 선언합니다.  
class Student:  
    pass  
  
# 학생을 선언합니다.  
student = Student()  
  
# 학생 리스트를 선언합니다.  
students = [  
    Student(),  
    Student(),  
    Student(),  
    Student(),  
    Student(),  
    Student()  
]
```

생성자

- 생성자(constructor)

- 클래스 이름과 같은 함수

```
class 클래스 이름:  
    def __init__(self, 추가적인 매개변수):  
        pass
```

- 클래스 내부의 함수는 첫 번째 매개변수로 반드시 self 입력해야 함
 - self : '자기 자신' 나타내는 디렉터리
 - self.<식별자> 형태로 접근

생성자

클래스를 선언합니다.

```
class Student:
```

```
    def __init__(self, name, korean, math, english, science):
```

```
        self.name = name
```

```
        self.korean = korean
```

```
        self.math = math
```

```
        self.english = english
```

```
        self.science = science
```

학생 리스트를 선언합니다.

```
students = [
```

```
    Student("윤인성", 87, 98, 88, 95),
```

```
    Student("연하진", 92, 98, 96, 98),
```

```
    Student("구지연", 76, 96, 94, 90),
```

```
    Student("나선주", 98, 92, 96, 92),
```

```
    Student("윤아린", 95, 98, 98, 98),
```

```
    Student("윤명월", 64, 88, 92, 92)
```

```
]
```

Student 인스턴스의 속성에 접근하는 방법

```
students[0].name
```

```
students[0].korean
```

```
students[0].math
```

```
students[0].english
```

```
students[0].science
```

메소드

- 메소드(method)

- 클래스가 가지고 있는 함수

```
class 클래스 이름:  
    def 메소드 이름(self, 추가적인 매개변수):  
        pass
```

메소드

- 예시 – 클래스 내부에 함수(메소드) 선언하기

```
01  # 클래스를 선언합니다.
02  class Student:
03      def __init__(self, name, korean, math, english, science):
04          self.name = name
05          self.korean = korean
06          self.math = math
07          self.english = english
08          self.science = science
09
10      def get_sum(self):
11          return self.korean + self.math + \
12                 self.english + self.science
13
14      def get_average(self):
15          return self.get_sum() / 4
16
```


메소드

```
17     def to_string(self):
18         return "{}\t{}\t{}".format(\
19             self.name,\
20             self.get_sum(),\
21             self.get_average())
22
23 # 학생 리스트를 선언합니다.
24 students = [
25     Student("윤인성", 87, 98, 88, 95),
26     Student("연하진", 92, 98, 96, 98),
27     Student("구지연", 76, 96, 94, 90),
28     Student("나선주", 98, 92, 96, 92),
29     Student("윤아린", 95, 98, 98, 98),
30     Student("윤명월", 64, 88, 92, 92)
31 ]
32
33 # 학생을 한 명씩 반복합니다.
34 print("이름", "총점", "평균", sep="\t")
35 for student in students:
36     # 출력합니다.
37     print(student.to_string())
```

실행결과		
이름	총점	평균
윤인성	368	92.0
연하진	384	96.0
구지연	356	89.0
나선주	378	94.5
윤아린	389	97.25
윤명월	336	84.0

2. 클래스의 추가적인 구문

시작하기 전에

- 상속
 - 어떤 클래스를 기반으로 그 속성과 기능을 물려받아 새로운 클래스 만드는 것
- isinstance() 함수
 - 상속 관계에 따라서 객체가 어떤 클래스를 기반으로 만들었는지 확인할 수 있게 해주는 함수
- str() 함수

어떤 클래스의 인스턴스인지 확인하기

- isinstance() 함수

- 객체가 어떤 클래스로부터 만들어졌는지 확인

```
isinstance(인스턴스, 클래스)
```

```
# 클래스를 선언합니다.  
class Student:  
    def __init__(self):  
        pass  
  
# 학생을 선언합니다.  
student = Student()  
  
# 인스턴스 확인하기  
print("isinstance(student, Student):", isinstance(student, Student))
```

```
isinstance(students[0], Student): True
```

어떤 클래스의 인스턴스인지 확인하기

- isinstance() 함수의 다양한 활용

- 리스트 내부에 여러 종류의 인스턴스 들어있을 때, 인스턴스들을 구분하며 속성과 기능 사용
- 예시 - isinstance() 함수 활용

```
01  # 학생 클래스를 선언합니다.
02  class Student:
03      def study(self):
04          print("공부를 합니다.")
05
06  # 선생님 클래스를 선언합니다.
07  class Teacher:
08      def teach(self):
09          print("학생을 가르칩니다.")
10
11  # 교실 내부의 객체 리스트를 생성합니다.
12  classroom = [Student(), Student(), Teacher(), Student(), Student()]
13
14  # 반복을 적용해서 적절한 함수를 호출하게 합니다.
15  for person in classroom:
16      if isinstance(person, Student):
17          person.study()
18      elif isinstance(person, Teacher):
19          person.teach()
```

실행결과

공부를 합니다.
공부를 합니다.
학생을 가르칩니다.
공부를 합니다.
공부를 합니다.

특수한 이름의 메소드

- 다양한 보조 기능들
 - `__<이름>__()` 형태
 - 특수한 상황에 자동으로 호출되도록 만들어짐

특수한 이름의 메소드

- 예시 - __str__() 함수

```
01  # 클래스를 선언합니다.
02  class Student:
03      def __init__(self, name, korean, math, english, science):
04          self.name = name
05          self.korean = korean
06          self.math = math
07          self.english = english
08          self.science = science
09
10      def get_sum(self):
11          return self.korean + self.math + \
12              self.english + self.science
```

특수한 이름의 메소드

```
13
14     def get_average(self):
15         return self.get_sum() / 4
16
17     def __str__(self):
18         return "{}\t{}\t{}".format(
19             self.name,
20             self.get_sum(),
21             self.get_average())
22
23 # 학생 리스트를 선언합니다.
24 students = [
25     Student("윤인성", 87, 98, 88, 95),
26     Student("연하진", 92, 98, 96, 98),
27     Student("구지연", 76, 96, 94, 90),
```



→ `__str__()`이라는 이름으로
함수를 선언했습니다.

특수한 이름의 메소드

```
28     Student("나선주", 98, 92, 96, 92),
29     Student("윤아린", 95, 98, 98, 98),
30     Student("윤명월", 64, 88, 92, 92)
31 ]
32
33 # 출력합니다.
34 print("이름", "총점", "평균", sep="\t")
35 for student in students:
36     print(str(student))
```

→ str() 함수의 매개변수로 넣으면
student의 __str__() 함수가 호출됩니다.

실행 결과		
이름	총점	평균
윤인성	368	92.0
연하진	384	96.0
구지연	356	89.0
나선주	378	94.5
윤아린	389	97.25
윤명월	336	84.0

이와 같이 __str__() 함수 정의하면 str() 함수 호출할 때 __str__() 함수가 자동으로 호출

클래스 변수와 메소드

- 클래스 변수

- class 구문 바로 아래의 단계에 변수를 선언

```
class 클래스 이름:  
    클래스 변수 = 값
```

- 클래스 변수에 접근

```
클래스 이름.변수 이름
```

클래스 변수와 메소드

- 예시 - 클래스 변수

```
01  # 클래스를 선언합니다.
02  class Student:
03      count = 0
04
05      def __init__(self, name, korean, math, english, science):
06          # 인스턴스 변수 초기화
07          self.name = name
08          self.korean = korean
09          self.math = math
10          self.english = english
11          self.science = science
12
13      # 클래스 변수 설정
```

실행결과

1번째 학생이 생성되었습니다.
2번째 학생이 생성되었습니다.
3번째 학생이 생성되었습니다.
4번째 학생이 생성되었습니다.
5번째 학생이 생성되었습니다.
6번째 학생이 생성되었습니다.

현재 생성된 총 학생 수는 6명입니다.

클래스 변수와 메소드

```
14         Student.count += 1
15         print("{}번째 학생이 생성되었습니다.".format(Student.count))
16
17     # 학생 리스트를 선언합니다.
18     students = [
19         Student("윤인성", 87, 98, 88, 95),
20         Student("연하진", 92, 98, 96, 98),
21         Student("구지연", 76, 96, 94, 90),
22         Student("나선주", 98, 92, 96, 92),
23         Student("윤아린", 95, 98, 98, 98),
24         Student("윤명월", 64, 88, 92, 92)
25     ]
26
27     # 출력합니다.
28     print()
29     print("현재 생성된 총 학생 수는 {}명입니다.".format(Student.count))
```

클래스 내부와 외부에서
클래스 변수에 접근할 때는
모두 `Student.count` 형태
(클래스이름.변수이름)를
사용합니다.

클래스 변수와 메소드

- 클래스 함수

- 클래스가 가진 함수
- '클래스가 가진 기능' 명시적으로 나타냄
- 데코레이터(decorator) : @classmethod

클래스 함수 만들기

```
class 클래스 이름:
    @classmethod
    def 클래스 함수(cls, 매개변수):
        pass
```

원하는 이름을 사용해도 되지만, 관례적으로 cls를 사용합니다.

클래스 함수 호출하기

클래스 이름.함수 이름(매개변수)

클래스 변수와 메소드

- 활용 예시 – Student.print()

```
01  # 클래스를 선언합니다.
02  class Student:
03      # 클래스 변수
04      count = 0
05      students = []
06
07      # 클래스 함수
08      @classmethod
09      def print(cls): → Student 클래스에 print() 함수를 구현합니다.
10          print("----- 학생 목록 -----")
11          print("이름\t총점\t평균")
12          for student in cls.students:
13              print(str(student))
14          print("-----")
15
```

→ Student.students라고 해도 상관없지만,
여기서는 매개변수로 받은 cls를 활용합니다.

클래스 변수와 메소드

```
16     # 인스턴스 함수
17     def __init__(self, name, korean, math, english, science):
18         self.name = name
19         self.korean = Korean
20         self.math = math
21         self.english = English
22         self.science = science
23         Student.count += 1
24         Student.students.append(self)
25
26     def get_sum(self):
27         return self.korean + self.math + \
28             self.english + self.science
29
30     def get_average(self):
```

클래스 변수와 메소드

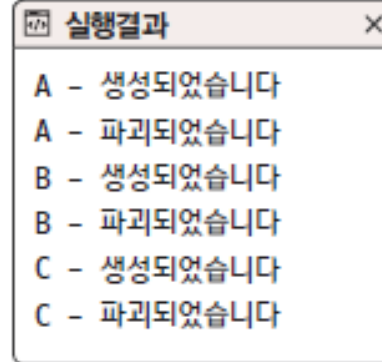
```
31         return self.get_sum() / 4
32
33     def __str__(self):
34         return "{}\t{}\t{}".format(\
35             self.name,\
36             self.get_sum(),\
37             self.get_average())
38
39     # 학생 리스트를 선언합니다.
40     Student("윤인성", 87, 98, 88, 95)
41     Student("연하진", 92, 98, 96, 98)
42     Student("구지연", 76, 96, 94, 90)
43     Student("나선주", 98, 92, 96, 92)
44     Student("윤아린", 95, 98, 98, 98)
45     Student("윤명월", 64, 88, 92, 92)
46     Student("김미화", 82, 86, 98, 88)
47     Student("김연화", 88, 74, 78, 92)
48     Student("박아현", 97, 92, 88, 95)
49     Student("서준서", 45, 52, 72, 78)
50
51     # 현재 생성된 학생을 모두 출력합니다.
52     Student.print()
```

실행결과		
----- 학생 목록 -----		
이름	총점	평균
윤인성	368	92.0
연하진	384	96.0
구지연	356	89.0
나선주	378	94.5
윤아린	389	97.25
윤명월	336	84.0
김미화	354	88.5
김연화	332	83.0
박아현	372	93.0
서준서	247	61.75

가비지 컬렉터

- 가비지 컬렉터(garbage collector)
 - 더 사용할 가능성이 없는 데이터를 메모리에서 제거하는 역할
 - 예시 – 가비지 컬렉터: 변수에 저장하지 않은 경우

```
01 class Test:
02     def __init__(self, name):
03         self.name = name
04         print("{} - 생성되었습니다".format(self.name))
05     def __del__(self):
06         print("{} - 파괴되었습니다".format(self.name))
07
08 Test("A")
09 Test("B")
10 Test("C")
```



실행결과

- A - 생성되었습니다
- A - 파괴되었습니다
- B - 생성되었습니다
- B - 파괴되었습니다
- C - 생성되었습니다
- C - 파괴되었습니다

가비지 컬렉터

- 예시 - 가비지 컬렉터: 변수에 저장한 경우

```
01 class Test:
02     def __init__(self, name):
03         self.name = name
04         print("{} - 생성되었습니다".format(self.name))
05     def __del__(self):
06         print("{} - 파괴되었습니다".format(self.name))
07
08 a = Test("A")
09 b = Test("B")
10 c = Test("C")
```

실행결과

A - 생성되었습니다
B - 생성되었습니다
C - 생성되었습니다
A - 파괴되었습니다
B - 파괴되었습니다
C - 파괴되었습니다

프라이빗 변수와 게터/세터

- 예시 - 원의 둘레와 넓이를 구하는 객체 지향 프로그램

```
01  # 모듈을 가져옵니다.  
02  import math  
03  
04  # 클래스를 선언합니다.  
05  class Circle:  
06      def __init__(self, radius):  
07          self.radius = radius  
08      def get_circumference(self):  
09          return 2 * math.pi * self.radius  
10      def get_area(self):  
11          return math.pi * (self.radius ** 2)  
12  
13  # 원의 둘레와 넓이를 구합니다.  
14  circle = Circle(10)  
15  print("원의 둘레:", circle.get_circumference())  
16  print("원의 넓이:", circle.get_area())
```

실행 결과

원의 둘레: 62.83185307179586
원의 넓이: 314.1592653589793

프라이빗 변수와 게터/세터

- 만약 Circle 클래스의 radius 속성에 다음과 같이 음수를 넣으면?
 - 원의 넓이는 제곱해서 양수, 둘레는 음수가 나오므로 현실과 맞지 않음
 - 따라서 길이를 음수로 넣는 것을 막는 방법이 필요

```
# 원의 둘레와 넓이를 구합니다.
```

```
circle = Circle(10)
```

```
circle.radius = -2
```

```
print("원의 둘레:", circle.get_circumference())
```

```
print("원의 넓이:", circle.get_area())
```

프라이빗 변수와 게터/세터

- 프라이빗 변수

- 변수를 마음대로 사용하는 것 방지
- __<변수 이름> 형태로 인스턴스 변수 이름 선언

```
01  # 모듈을 가져옵니다.
02  import math
03
04  # 클래스를 선언합니다.
05  class Circle:
06      def __init__(self, radius):
07          self.__radius = radius
08      def get_circumference(self):
09          return 2 * math.pi * self.__radius
10      def get_area(self):
11          return math.pi * (self.__radius ** 2)
12
```

프라이빗 변수와 게터/세터

```
13  # 원의 둘레와 넓이를 구합니다.
14  circle = Circle(10)
15  print("# 원의 둘레와 넓이를 구합니다.")
16  print("원의 둘레:", circle.get_circumference())
17  print("원의 넓이:", circle.get_area())
18  print()
19
20  # __radius에 접근합니다.
21  print("# __radius에 접근합니다.")
22  print(circle.__radius)
```

실행결과

원의 둘레와 넓이를 구합니다.

원의 둘레: 62.83185307179586

원의 넓이: 314.1592653589793

__radius에 접근합니다.

Traceback (most recent call last):

File "private_var.py", line 22, in <module>

print(circle.__radius)

AttributeError: 'Circle' object has no attribute '__radius'

프라이빗 변수와 게터/세터

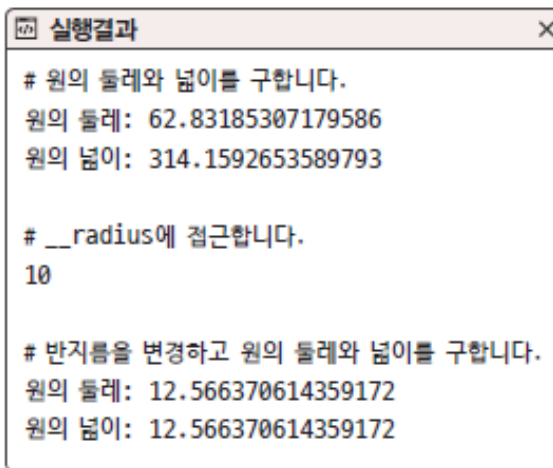
- 게터(getter)와 세터(setter)

- 프라이빗 변수 값 추출하거나 변경할 목적으로 간접적으로 속성에 접근하도록 하는 함수
- 예시

```
01  # 모듈을 가져옵니다.
02  import math
03
04  # 클래스를 선언합니다.
05  class Circle:
06      def __init__(self, radius):
07          self.__radius = radius
08      def get_circumference(self):
09          return 2 * math.pi * self.__radius
10      def get_area(self):
11          return math.pi * (self.__radius ** 2)
12
```

프라이빗 변수와 게터/세터

```
13     # 게터와 세터를 선언합니다.
14     def get_radius(self):
15         return self.__radius
16     def set_radius(self, value):
17         self.__radius = value
18
19     # 원의 둘레와 넓이를 구합니다.
20     circle = Circle(10)
21     print("# 원의 둘레와 넓이를 구합니다.")
22     print("원의 둘레:", circle.get_circumference())
23     print("원의 넓이:", circle.get_area())
24     print()
25
26     # 간접적으로 __radius에 접근합니다.
27     print("# __radius에 접근합니다.")
28     print(circle.get_radius())
29     print()
30
31     # 원의 둘레와 넓이를 구합니다.
32     circle.set_radius(2)
33     print("# 반지름을 변경하고 원의 둘레와 넓이를 구합니다.")
34     print("원의 둘레:", circle.get_circumference())
35     print("원의 넓이:", circle.get_area())
```



실행결과

```
# 원의 둘레와 넓이를 구합니다.
원의 둘레: 62.83185307179586
원의 넓이: 314.1592653589793

# __radius에 접근합니다.
10

# 반지름을 변경하고 원의 둘레와 넓이를 구합니다.
원의 둘레: 12.566370614359172
원의 넓이: 12.566370614359172
```


프라이빗 변수와 게터/세터

- 이와 같이 함수 사용해 값 변경하면 여러 가지 처리 추가할 수 있음
 - ex) set_radius() 함수에 다음과 같은 코드 추가하여 __radius에 할당할 값을 양의 숫자로만 한정

```
def set_radius(self, value):  
    if value <= 0:  
        raise TypeError("길이는 양의 숫자여야 합니다.")  
    self.__radius = value
```

프라이빗 변수와 게터/세터

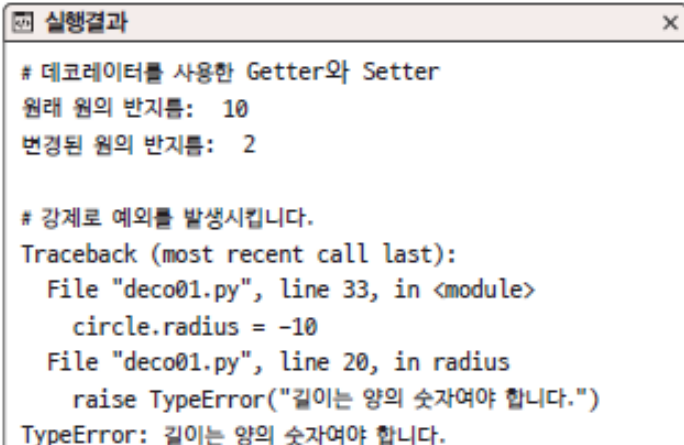
- 데코레이터를 사용한 게터와 세터

- 파이썬 프로그래밍 언어에서 제공하는 게터와 세터 만들고 사용하는 기능
 - 변수 이름과 같은 함수 정의하고 위에 @property와 @<변수 이름>.setter 데코레이터 붙이기
 - 주의) 파이썬에선 게터를 만들지 않으면 세터를 만들 수 없음

```
01  # 모듈을 가져옵니다.
02  import math
03
04  # 클래스를 선언합니다.
05  class Circle:
    # ...생략...
13  # 게터와 세터를 선언합니다.
14  @property
15  def radius(self):
16      return self.__radius
17  @radius.setter
18  def radius(self, value):
19      if value <= 0:
20          raise TypeError("길이는 양의 숫자여야 합니다.")
```

프라이빗 변수와 게터/세터

```
21         self.__radius = value
22
23     # 원의 둘레와 넓이를 구합니다.
24     print("# 데코레이터를 사용한 Getter와 Setter")
25     circle = Circle(10)
26     print("원래 원의 반지름: ", circle.radius)
27     circle.radius = 2
28     print("변경된 원의 반지름: ", circle.radius)
29     print()
30
31     # 강제로 예외를 발생시킵니다.
32     print("# 강제로 예외를 발생시킵니다.")
33     circle.radius = -10
```



```
실행결과
# 데코레이터를 사용한 Getter와 Setter
원래 원의 반지름:  10
변경된 원의 반지름:  2

# 강제로 예외를 발생시킵니다.
Traceback (most recent call last):
  File "deco01.py", line 33, in <module>
    circle.radius = -10
  File "deco01.py", line 20, in radius
    raise TypeError("길이는 양의 숫자여야 합니다.")
TypeError: 길이는 양의 숫자여야 합니다.
```

상속

- 상속(inheritance)
 - 다른 누군가가 만든 기본 형태에 내가 원하는 것만 추가하거나 교체
- 다중 상속
 - 다른 누군가가 만든 형태들을 조립하여 내가 원하는 것을 만드는 것
- 프로그래밍 언어에서 기반이 되는 것을 부모, 이를 기반으로 생성한 것을 자식이라 부름. 부모가 자식에게 자신의 기반을 물려주는 기능이므로 "상속"

상속

```
01  # 부모 클래스를 선언합니다.
02  class Parent:
03      def __init__(self):
04          self.value = "테스트"
05          print("Parent 클래스의 __init()__ 메소드가 호출되었습니다.")
06      def test(self):
07          print("Parent 클래스의 test() 메소드입니다.")
08
09  # 자식 클래스를 선언합니다.
10  class Child(Parent):
11      def __init__(self):
12          super().__init__() # 부모의 __init__() 함수를 호출합니다.
13          print("Child 클래스의 __init()__ 메소드가 호출되었습니다.")
14
15  # 자식 클래스의 인스턴스를 생성하고 부모의 메소드를 호출합니다.
16  child = Child()
17  child.test()
18  print(child.value)
```

실행 결과

```
Parent 클래스의 __init()__ 메소드가 호출되었습니다.
Child 클래스의 __init()__ 메소드가 호출되었습니다.
Parent 클래스의 test() 메소드입니다.
테스트
```

상속

- 예외 클래스 만들기

- Exception 클래스 수정하여 CustomException 클래스 만들기
- 예시 - 사용자 정의 예외 클래스 만들기

```
01 class CustomException(Exception):  
02     def __init__(self):  
03         super().__init__()  
04  
05 raise CustomException
```

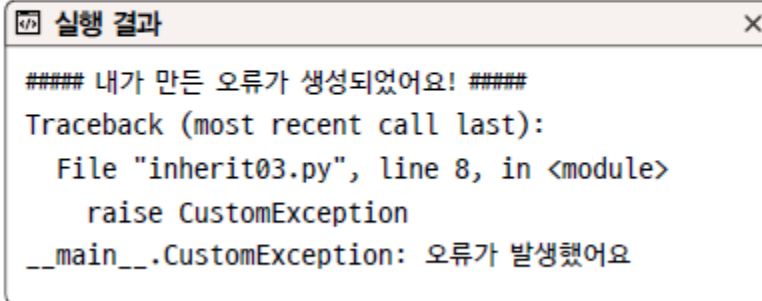
 실행 결과

Traceback (most recent call last):
File "inherit02.py", line 5, in <module>
raise CustomException
__main__.CustomException

상속

- 예시 - 자식 클래스로써 부모의 함수 재정의(오버라이드)하기

```
01 class CustomException(Exception):
02     def __init__(self):
03         super().__init__()
04         print("##### 내가 만든 오류가 생성되었어요! #####")
05     def __str__(self):
06         return "오류가 발생했어요"
07
08 raise CustomException
```



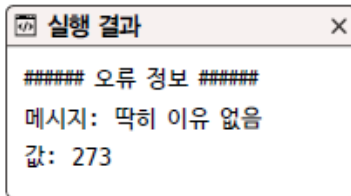
실행 결과

```
##### 내가 만든 오류가 생성되었어요! #####
Traceback (most recent call last):
  File "inherit03.py", line 8, in <module>
    raise CustomException
__main__.CustomException: 오류가 발생했어요
```

상속

- 예시 – 자식 클래스로써 부모에 없는 새로운 함수 정의하기

```
01  # 사용자 정의 예외를 생성합니다.
02  class CustomException(Exception):
03      def __init__(self, message, value):
04          super().__init__()
05          self.message = message
06          self.value = value
07
08      def __str__(self):
09          return self.message
10
11      def print(self):
12          print("##### 오류 정보 #####")
13          print("메시지:", self.message)
14          print("값:", self.value)
15  # 예외를 발생시켜 봅니다.
16  try:
17      raise CustomException("딱히 이유 없음", 273)
18  except CustomException as e:
19      e.print()
```

실행 결과 창은 제목 '실행 결과'와 닫기 버튼 'X'를 가지고 있습니다. 창 내부에는 다음과 같은 텍스트가 표시되어 있습니다:

```
##### 오류 정보 #####
메시지: 딱히 이유 없음
값: 273
```